

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Домашняя работа №4

Технический дизайн микросервисной архитектуры сервиса бронирования
ресторанов

Выполнил:

Ермаков Максим Олегович

Группа К3340

Проверил:

Добряков Д. И.

Санкт-Петербург
2026 г.

Задача

В рамках домашней работы необходимо спроектировать переход монолитного backend-приложения сервиса бронирования ресторанов к микросервисной архитектуре с соблюдением подхода database-per-service.

- Разделить монолитное backend-приложение на микросервисы по предметным границам.
- Описать архитектуру, ответственность сервисов и способы их взаимодействия.
- Спроектировать разделение единой базы данных на обособленные базы данных сервисов.
- Описать внутренние endpoint-ы для межсервисного взаимодействия в формате OpenAPI.
- Зафиксировать варианты запросов, ответов и возможных ошибок.
- Сформировать требования и шаги перехода от монолита к микросервисам.

Ход работы

Анализ исходного монолита

Исходное приложение реализует регистрацию и вход пользователей, JWT-авторизацию, справочники, поиск ресторанов, карточку ресторана, меню, фотографии, отзывы, столики и бронирования. В монолитной реализации все эти области связаны общей кодовой базой и единой PostgreSQL-БД, что усложняет независимое развитие и масштабирование отдельных частей системы.

Проектируемые компоненты

Компонент	Ответственность	База данных
api-gateway	Единая публичная точка входа, проверка авторизации, маршрутизация и композиция сложных ответов	Нет
identity-service	Пользователи, роли, регистрация, вход, выпуск и проверка JWT	identity_db
catalog-service	Рестораны, локации, кухни, фотографии, публикация и денормализованный рейтинг	catalog_db
menu-service	Категории меню и позиции меню ресторанов	menu_db
reservation-service	Столики, доступность, создание и жизненный цикл бронирований	reservation_db
review-service	Отзывы пользователей, проверка уникальности отзыва и расчет рейтинга	review_db

Публичные клиенты обращаются только к api-gateway. Gateway сохраняет внешний контракт /api/v1, проверяет токен через identity-service и передает во внутренние сервисы доверенный пользовательский контекст в заголовках X-User-Id, X-User-Role и X-Request-Id.

Разделение баз данных

БД	Владелец	Основные таблицы
identity_db	identity-service	users, refresh_sessions
catalog_db	catalog-service	locations, cuisines, restaurants, restaurant_cuisines, restaurant_photos
menu_db	menu-service	menu_categories, menu_items
reservation_db	reservation-service	restaurant_tables, reservations
review_db	review-service	reviews

Сервисы не создают внешние ключи на таблицы чужих БД. Например, reservation-service хранит user_id и restaurant_id как внешние идентификаторы, но проверяет пользователя через identity-service, а ресторан через catalog-service. Такой подход сохраняет автономность данных каждого сервиса.

Межсервисное взаимодействие

Сценарий	Взаимодействие
Проверка авторизации	api-gateway вызывает POST /internal/auth/introspect в identity-service и получает userId, role, email
Создание бронирования	reservation-service проверяет пользователя через identity-service и опубликованный ресторан через catalog-service
Карточка ресторана	api-gateway собирает данные из catalog-service, menu-service, reservation-service и review-service
Создание отзыва	review-service проверяет ресторан через catalog-service, сохраняет отзыв и обновляет рейтинг ресторана
Административные операции	gateway проверяет роль ADMIN и маршрутизирует запрос в сервис-владелец агрегата

Внутренний OpenAPI-контракт

Для межсервисных вызовов подготовлена спецификация OpenAPI в файле internal-openapi.yaml. В ней описаны внутренние операции identity-service, catalog-service, menu-service, reservation-service и review-service, форматы запросов и ответов, а также типовые ошибки.

Группа endpoint-ов	Назначение
/internal/auth/introspect	Проверка access token и возврат текущего пользователя
/internal/users/{userId}	Получение краткого профиля пользователя для отзывов и бронирований
/internal/restaurants/{restaurantId}	Проверка существования и публикации ресторана
/internal/restaurants/{restaurantId}/rating-summary	Обновление денормализованного рейтинга ресторана

Группа endpoint-ов	Назначение
/internal/restaurants/{restaurantId}/menu	Получение меню ресторана
/internal/restaurants/{restaurantId}/availability	Проверка доступности столиков

План перехода

- Выделить identity-service и перенести пользователей, роли, регистрацию, вход и JWT-проверку.
- Выделить catalog-service и перенести справочники, рестораны, кухни, локации и фотографии.
- Выделить menu-service, reservation-service и review-service с собственными БД.
- Сохранить публичный API через api-gateway и постепенно заменить прямые обращения монолита внутренними REST-вызовами.
- Перенести данные миграциями, проверить ключевые сценарии smoke-тестом и только после этого отключать монолитные участки.

Вывод

В результате работы спроектирована микросервисная архитектура сервиса бронирования ресторанов. Были определены границы сервисов, правила владения данными, схема database-per-service, внутренние REST-контракты и последовательность перехода от монолита к набору независимых backend-сервисов. Подготовленный дизайн является основой для реализации разделения в лабораторной работе №2.